

Relazione progetto WASMIDIP per il corso "Linguaggi di programmazione"

Leonardo Ganzaroli (657247)

Gennaio 2026

Obiettivo

"Progettare ed implementare un sistema software per convertire e suonare una melodia MIDI (...) in WASM¹ scrivendo codice Rust"

Processo di sviluppo

Lo sviluppo generale, ovviamente, non è partito immediatamente con la scrittura del codice ma è stato invece necessario svolgere una ricerca preliminare atta a capire meglio come procedere. In particolare i primi 3 quesiti da chiarire sono stati:

- Che cos'è il WebAssembly?
- Come si effettua la compilazione Rust→WASM?
- Come si può riprodurre un file MIDI in Rust?

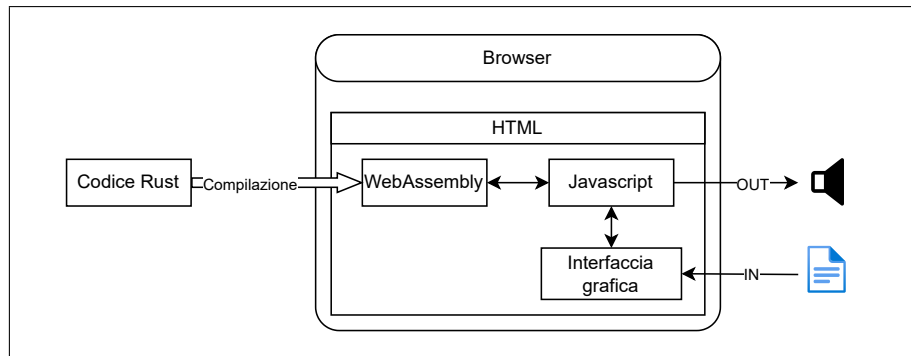
Dopo una breve ricerca è stato possibile trovare delle risposte, per i primi due punti è bastato individuare del materiale teorico [1] e per il restante esistono vari crate già pronti [2, 3, 4] per l'elaborazione e la successiva riproduzione dei file MIDI. Gli esempi presenti in quest'ultimi si sono inoltre rivelati un'ottima base con cui modellare le fondamenta dell'intero sistema, è infatti bastato fare delle leggere modifiche agli stessi e recuperare dei file MIDI per creare un semplice player.

Una volta appurato il funzionamento di questo piccolo prototipo è stato possibile passare alla fase successiva: trasformare il tutto in WebAssembly ed utilizzarlo tramite un browser. Questo ha però introdotto un problema fondamentale, il metodo usato fin'ora non può funzionare in WebAssembly dato che all'atto pratico quest'ultimo viene eseguito all'interno di una sandbox e l'unico modo che ha per comunicare con l'ambiente esterno è tramite JavaScript [5].

¹Abbreviazione di WebAssembly.

Fortunatamente resta possibile mantenere la parte di elaborazione dei file, che risulta essere perfetta da svolgere in WebAssembly essendo la parte più computazionalmente intensiva. La parte di I/O invece viene delegata a JavaScript (e HTML).

Con queste nuove informazioni si può abbozzare uno schema:



Schema generale del sistema

Nel dettaglio:

Parte HTML : contiene tutti gli altri elementi e implementa l'interfaccia grafica, data la sua semplicità è stata creata totalmente da 0 senza aiuti esterni;

Parte Rust e WASM : crea il buffer audio partendo dal file e fai dei controlli sulla struttura di quest'ultimo per garantire che rispetti lo standard [6], l'implementazione è stata ottenuta modificando la parte "salvabile" del prototipo menzionato prima;

Parte JS² : implementa il comportamento dell'interfaccia, fa da intermediario tra gli altri componenti e riproduce l'audio basandosi sul buffer ricevuto dalla parte WASM. Questa parte è stata la più complessa da implementare, non tanto per la parte audio [7] ma per il fatto che bisognava interfacciare il tutto con altri 2 linguaggi;

A livello pratico il flusso di funzionamento si può riassumere come:

1. L'utente inserisce un file tramite l'apposito elemento HTML
2. JS lo passa a WASM sottoforma di array di byte
3. WASM crea un buffer, lo popola usando il file e lo rimanda indietro
4. JS trasforma il buffer ricevuto in un `AudioBuffer`
5. L'utente può adesso controllare la riproduzione tramite l'interfaccia o inserire un nuovo file e ricominciare da capo

²Abbreviazione di JavaScript.

Resta però un ultimo problema tecnico da risolvere, infatti i browser non eseguono il codice WASM presente nei file aperti direttamente dal disco. Per aggirare la limitazione bisogna necessariamente farglieli spedire da un server HTTP [8]. Questa si è rivelata solamente una breve seccatura dato che esistono metodi molto semplici per crearne uno adatto a questo contesto³ [9].

Considerazione sui paradigmi usati

Paradigma	Dove?
Imperativo/Procedurale	Elaborazione e controllo del file MIDI
Scripting	Compilazione del codice Rust e creazione del server HTTP ⁴
Orientato agli oggetti	Astrazione del player MIDI come classe in JS e istanziazione di classi già presenti
Orientato agli eventi	Implementazione comportamento dell'interfaccia grafica

Riguardo l'uso di I.A.

Tutti i vari esempi/articoli/tutorial/ecc... forniti dalla traccia o trovati su internet sono stati più che sufficienti per creare il sistema facendo una specie di collage, quindi nell'ambito del codice (nonostante non avessi conoscenze pregresse né di Rust né di JavaScript) non è sorto il problema di per se. Si sono però presentati 2 momenti in cui sono apparsi dei problemi esterni e mi sono avvalso di ChatGPT per cercare di capirne le cause in maniera rapida. Prompt utilizzati:

1. Perché se carico una pagina HTML contenente codice WebAssembly direttamente dal disco non funziona correttamente?
2. Se ho una pagina HTML passata da un server HTTP e faccio ripartire quest'ultimo, perché ogni tanto non funziona la parte JavaScript se ricarico la pagina invece di chiuderla e riaprirla?

³Questo metodo va bene SOLAMENTE per progetti piccoli come questo, non per vere applicazioni.

⁴Per risolvere alcuni problemi riscontrati durante l'esecuzione il server è stato configurato in modo da comunicare al browser di non utilizzare la cache per questa pagina.

Riferimenti bibliografici

- [1] MDN Web Docs, <https://developer.mozilla.org/en-US/>, Sezioni interessate: WebAssembly e JavaScript.
- [2] insomnimus, “nodi,” <https://github.com/insomnimus/nodi>.
- [3] sinshu, “rustysynth,” <https://github.com/sinshu/rustysynth>.
- [4] RustAudio, “rodio,” <https://github.com/RustAudio/rodio>.
- [5] D. Elm, “What WebAssembly is Not,” <https://learn-wasm.dev/tutorial/introduction/what-webassembly-is-not>.
- [6] *Standard MIDI Files 1.0*, The MIDI Manufacturers Association, Feb. 1996.
- [7] Boris Smus, “Getting started with Web Audio API,” <https://web.dev/articles/webaudio-intro>.
- [8] RemObjects Software, “Deploying Web Modules,” <https://docs.elementcompiler.com/Platforms/WebAssembly/Deployment/Web/>.
- [9] PHP Manual. <https://www.php.net/manual/en/features.commandline.webserver.php>.

Altro materiale usato

- [Come tenere traccia del tempo di riproduzione](#)
- [Tutorial HTML e CSS](#)
- [File\(s\) MIDI](#)
- [Soundfont scelto](#)